

Experimental Data Management System

Milestones 1–6: Evolution to an Intelligent Concurrent System

1. PROBLEM DEFINITION

1.1 Background

Experimental data analysis is widely used in scientific and industrial environments such as chemistry, agriculture, and quality control. These datasets often contain multiple variables that require structured storage, validation, and statistical analysis.

In this project, a wine quality dataset is used as a representative experimental dataset.

1.2 Problem Statement

Traditional tools such as spreadsheets and basic scripts are limited in handling:

Large-scale experimental datasets

Automated validation of data

Efficient computation of statistical metrics

Detection of abnormal or inconsistent values

Concurrent processing of large workloads

These limitations reduce efficiency and increase the risk of human error.

1.3 Research Gap

There is a lack of integrated systems that combine:

Object-oriented data modeling

Statistical analysis

Concurrent processing

Intelligent anomaly detection

Most existing solutions handle these features separately rather than as a unified system.

1.4 Proposed Solution

This project proposes an Experimental Data Management System (EDMS) that integrates:

Object-oriented design for data representation

Design patterns for scalability and maintainability

Concurrent processing for performance improvement

Statistical anomaly detection using Z-score analysis

2. SYSTEM EVOLUTION

The system was developed progressively across six milestones:

2.1 Milestone 1–2: Basic System Design

Introduction of object-oriented structure

Creation of core classes (Experiment, Trial, Measurement)

Basic data storage and retrieval

2.2 Milestone 3: Design Patterns Integration

Strategy Pattern for flexible validation

Observer Pattern for logging system events

Improved modularity and maintainability

2.3 Milestone 4: Functional Data Processing

Use of generators for memory-efficient iteration

Functional programming techniques (filter, lambda)

Introduction of statistical computations

2.4 Milestone 5: Concurrency and Performance

Multiprocessing for statistical computation (mean, variance)

Threading for task queue processing

Asynchronous file export using asyncio

Performance benchmarking (sequential vs parallel)

2.5 Milestone 6: Intelligent System Enhancement

Introduction of anomaly detection module

Z-score based statistical outlier detection

Classification of data into:

Normal

Borderline

Outlier

2.6 System Evolution Summary

The system progressed from a basic object-oriented data model to a concurrent analytics platform. The final version integrates anomaly detection and supports automated decision-making for large-scale datasets.

3. ARCHITECTURE AND DESIGN

3.1 System Architecture Overview

The system is designed using a layered architecture:

Layers:

Data Input Layer (CSV Loader)

Object Model Layer (Experiment, Trial, Measurement)

Processing Layer (Statistical Analysis)

Concurrency Layer (Threads, Multiprocessing, Async IO)

Intelligence Layer (Anomaly Detection)

Logging Layer (Observer Pattern)

3.2 Object-Oriented Design

Measurement: Represents a single data point

Trial: Groups multiple measurements

Experiment: Manages multiple trials and analysis

This ensures modularity and clear separation of responsibilities.

3.3 Design Patterns Used

Strategy Pattern

Used for flexible validation of measurement values.

Observer Pattern

Used for automatic logging of system events.

3.4 Architecture Diagram Description

The system flows as:

CSV Dataset → Loader → Validation → Experiment Model → Processing → Concurrency
→ Anomaly Detection → Output & Logging

4. TESTING AND EVALUATION

4.1 Functional Testing

The system was tested for:

Dataset loading

Validation of missing or invalid values

Statistical computation accuracy

Anomaly detection correctness

All core functions executed successfully.

4.2 Performance Testing

The system compares:

Sequential processing

Parallel (multiprocessing) execution

Result:

Parallel execution reduced computation time for large datasets.

4.3 Edge Case Testing

The system handles:

Missing values → raises `ValidationError`

Zero variance cases → safely handled in anomaly detection

Large datasets → processed using generators and multiprocessing

4.4 Evaluation Summary

The system demonstrates:

Correctness

Efficiency

Scalability

Robust error handling